

Karlova Univerzita
Přírodovědecká fakulta
Katedra aplikované geoinformatiky a kartografie



Algoritmy počítačové kartografie

Digitální model terénu

Aliaksandra Sparysh
Jolana Václavková

Praha 2026

Kapitola 1

Zadání

Úloha č. 3: Digitální model terénu

Vstup: množina $P = \{p_1, \dots, p_n\}$, $p_i = \{x_i, y_i, z_i\}$.

Výstup: polyedrický DMT nad množinou P představovaný vrstevnicemi doplněný vizualizací sklonu trojúhelníků a jejich expozicí.

Metodou inkrementální konstrukce vytvořte nad množinou P vstupních bodů 2D Delaunay triangulaci. Jako vstupní data použijte existující geodetická data (alespoň 300 bodů), popř. navrhnete algoritmus pro generování syntetických vstupních dat představujících významné terénní tvary (kupa, údolí, spočinek, hřbet, ...).

Vstupní množiny bodů včetně níže uvedených výstupů vhodně vizualizujte. Grafické rozhraní realizujte s využitím frameworku QT. Dynamické datové struktury implementujte s využitím STL.

Nad takto vzniklou triangulací vygenerujte polyedrický digitální model terénu. Dále proveďte tyto analýzy:

- S využitím lineární interpolace vygenerujte vrstevnice se zadaným krokem a v zadaném intervalu, proveďte jejich vizualizaci s rozlišením zvýrazněných vrstevnic.
- Analyzujte sklon digitálního modelu terénu, jednotlivé trojúhelníky vizualizujte v závislosti na jejich sklonu.
- Analyzujte expozici digitálního modelu terénu, jednotlivé trojúhelníky vizualizujte v závislosti na jejich expozici ke světové straně.

Zhodnoťte výsledný digitální model terénu z kartografického hlediska, zamyslete se nad slabinami algoritmu založeného na 2D Delaunay triangulaci. Ve kterých situacích (různé terénní tvary) nebude dávat vhodné výsledky? Tyto situace graficky znázorněte.

Zhodnocení činnosti algoritmu včetně ukázek proveďte alespoň na 3 strany formátu A4.

Kapitola 2

Údaje o bonusových úlohách

Krok	Hodnocení
Delaunay triangulace, polyedrický model terénu.	10b
Konstrukce vrstevnic, analýza sklonu a expozice.	10b
<i>Výběr barevných stupnic při vizualizaci sklonu a expozice.</i>	<i>+3b</i>
<i>Automatický popis vrstevnic.</i>	<i>+3b</i>
<i>Algoritmus pro automatické generování terénních tvarů (kupa, údolí, spočinek, hřbet, ...).</i>	<i>+10b</i>
<i>Zpracování textové zprávy v LaTeXu</i>	<i>+10b</i>
Max celkem:	46b

Kapitola 3

Popis a rozbor problému

3.1 Popis a rozbor problému

Realistické zobrazení zemského povrchu představuje vzhledem k jeho nepravidelnému charakteru poměrně komplikovaný proces. Povrch Země nelze jednoduše popsat elementárními geometrickými útvary, jelikož se jedná o složitý systém obsahující jak plynulé změny reliéfu, tak i ostré terénní zlomy. Modelování takového 3D prostředí je náročné nejen z hlediska matematického popisu, ale i z hlediska kartografických zásad, estetických požadavků a požadavků na efektivní výpočetní zpracování.

Z těchto důvodů se v současnosti využívají digitální modely terénu (DMT), které umožňují reprezentovat zemský povrch v digitální podobě a dále s ním pracovat. DMT představuje diskrétní aproximaci spojitého povrchu, přičemž kromě samotné reprezentace výšky umožňuje provádět i řadu analýz, jako je výpočet sklonu, orientace svahu, generování vrstevnic nebo vizualizace terénu pomocí hypsometrie či stínování.

3.1.1 Typy digitálních modelů

V literatuře se rozlišuje několik typů digitálních modelů, které se liší zejména rozsahem reprezentovaných dat:

- Digitální model terénu (DMT) – reprezentuje pouze holý zemský povrch bez umělých objektů, jako jsou budovy či vegetace.
- Digitální model povrchu (DMP) – zahrnuje kromě terénu i objekty nacházející se na jeho povrchu.
- Digitální výškový model (DVM) – pracuje výhradně s nadmořskými výškami bodů bez další interpretace.

V zahraniční literatuře se často používá termín DEM (Digital Elevation Model), který může označovat kterýkoliv z výše uvedených modelů v závislosti na kontextu.

Digitální modely terénu nacházejí široké uplatnění v řadě oblastí, jako je plánování dopravní infrastruktury, analýza přírodních rizik (např. sesuvy půdy nebo eroze), hydrologie, lesnictví, vojenské aplikace nebo počítačová grafika.

3.1.2 Reprezentace terénu

V praxi se využívají tři základní způsoby reprezentace terénu:

- rastrový model (grid),
- trojúhelníkový model (TIN),
- plátový model.

V této práci je použit trojúhelníkový model terénu (TIN – Triangulated Irregular Network), který reprezentuje povrch pomocí nepravidelné sítě trojúhelníků. Tento model umožňuje efektivně zachytit složité tvary reliéfu při menším množství dat než rastrový model.

Základem modelu je množina bodů:

$$P = \{p_1, p_2, \dots, p_n\}, \quad p_i = (x_i, y_i, z_i),$$

kde (x_i, y_i) představují souřadnice bodu v rovině a z_i jeho výšku.

3.1.3 Triangulace

Triangulace představuje rozdělení roviny na množinu trojúhelníků tak, aby:

- se žádné dva trojúhelníky se nepřekrývaly,
- sdílely maximálně jednu hranu,
- žádný bod neležel uvnitř žádného trojúhelníku.

Výsledná triangulace je označena jako:

$$T = \{t_1, t_2, \dots, t_m\}.$$

Počet hran a trojúhelníků lze vyjádřit pomocí vztahů:

$$n_h = 3n - 3 - k, \quad n_t = 2n - 2 - k,$$

kde k je počet bodů na konvexní obálce množiny P .

3.1.4 Delaunayho triangulace

Pro konstrukci triangulace je využita Delaunayho triangulace, která je v oblasti GIS standardem. Je definována podmínkou prázdné kružnice:

Uvnitř kružnice opsané libovolnému trojúhelníku neleží žádný další bod množiny P .

Mezi hlavní vlastnosti Delaunayho triangulace patří:

- maximalizace minimálního úhlu v trojúhelnících,
- vznik geometricky kvalitních trojúhelníků,
- lokální i globální optimalita vzhledem k tomuto kritériu,
- jednoznačnost v případě, že žádné čtyři body neleží na jedné kružnici.

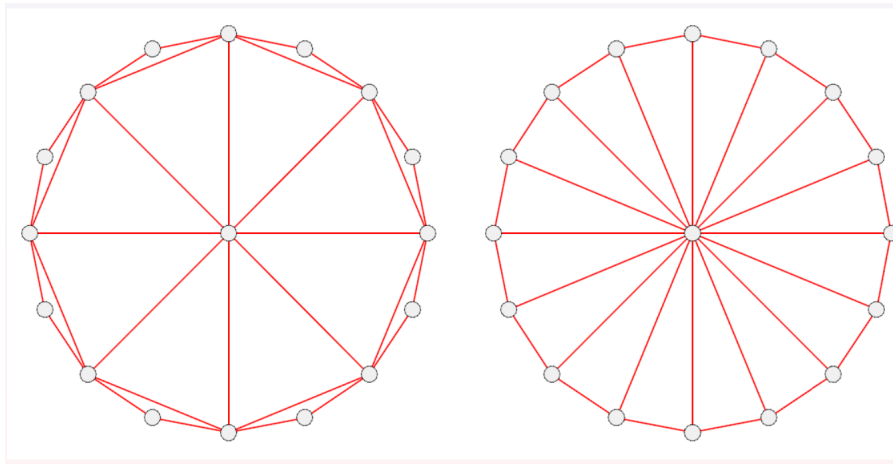
Tyto vlastnosti zajišťují stabilitu modelu a jeho vhodnost pro interpolaci.

3.1.5 Porovnání Delaunay a Greedy triangulace

Při konstrukci triangulace nad množinou bodů lze využít různé přístupy, které se liší svým principem i kvalitou výsledné sítě. Mezi základní patří Delaunayho triangulace a tzv. greedy triangulace.

- **Delaunayho triangulace** je definována podmínkou prázdné kružnice, která zajišťuje, že uvnitř kružnice opsané libovolnému trojúhelníku neleží žádný další bod množiny. Díky tomu maximalizuje minimální úhel v trojúhelnících, čímž vznikají geometricky kvalitní a relativně pravidelné prvky. Tato vlastnost je zásadní zejména pro interpolaci a modelování terénu.
- **Greedy triangulace** naopak vzniká postupným přidáváním nejkratších možných hran, které se navzájem nekříží. Tento přístup je jednoduchý a snadno implementovatelný, avšak nebere v úvahu celkovou strukturu triangulace.

Hlavní rozdíl spočívá v optimalizačním kritériu - zatímco Delaunayho triangulace optimalizuje tvar trojúhelníků, greedy triangulace minimalizuje délku hran. V důsledku toho může greedy triangulace vytvářet nevhodné, protáhlé trojúhelníky. A právě z tohoto důvodu je pro konstrukci digitálního modelu terénu vhodnější Delaunayho triangulace, která poskytuje stabilnější a přesnější reprezentaci povrchu.



Obrázek 3.1: Porovnání Greedy (vlevo) a Delaunay triangulace

3.1.6 Inkrementální konstrukce

Triangulace je vytvářena inkrementální metodou, která postupně přidává body do existující struktury.

Pro danou orientovanou hranu $e = (p_1, p_2)$ se hledá bod p , který:

- leží v levé polorovině vzhledem k hraně,
- maximalizuje úhel $\angle(p_1, p, p_2)$:

$$p = \arg \max_{p_i} \angle(p_1, p_i, p_2)$$

Tímto způsobem vznikají nové trojúhelníky, které jsou postupně přidávány do triangulace.

3.1.7 Polyedrický model terénu

Po vytvoření triangulace je model rozšířen do 3D prostoru. Každý trojúhelník definuje rovinu:

$$z = ax + by + c,$$

čímž vzniká polyedrický model terénu, který aproximuje skutečný povrch.

3.1.8 Konstrukce vrstevnic

Vrstevnice představují spojnice bodů se stejnou nadmořskou výškou. Jsou definovány jako průnik modelu s rovinami:

$$z = h.$$

Pro výpočet průsečíků se používá lineární interpolace:

$$\begin{aligned}x &= x_1 + \frac{z - z_1}{z_2 - z_1}(x_2 - x_1), \\y &= y_1 + \frac{z - z_1}{z_2 - z_1}(y_2 - y_1).\end{aligned}$$

Podmínka průniku je:

$$(z_1 - z)(z_2 - z) < 0.$$

3.1.9 Analýza sklonu

Sklon terénu je definován jako úhel mezi rovinou trojúhelníku a horizontální rovinou.

Pro rovinu:

$$\rho : ax + by + cz + d = 0$$

je normálový vektor:

$$\mathbf{n} = (a, b, c).$$

Sklon se vypočítá:

$$\varphi = \arccos \left(\frac{c}{\sqrt{a^2 + b^2 + c^2}} \right).$$

3.1.10 Analýza expozice

Expozice (orientace svahu) je určena projekcí gradientu do roviny xy :

$$\mathbf{v} = (a, b, 0).$$

Azimut expozice je:

$$A = \arctan \left(\frac{a}{b} \right).$$

Pro správný výpočet je vhodné použít funkci `atan2`, která zohledňuje kvadrant.

Kapitola 4

Popisy algoritmů formálním jazykem

4.1 Delaunay triangulace – inkrementální konstrukce

Pro konstrukci triangulace byla zvolena inkrementální metoda využívající seznam aktivních hran (Active Edge List, AEL). Tento přístup postupně buduje triangulaci přidáváním trojúhelníků na základě lokálního kritéria.

Algoritmus začíná výběrem počátečního bodu q jako bodu s minimální souřadnicí y :

```
q = min(points, key = lambda k: k.y())
```

Následně je nalezen nejbližší bod q_n pomocí eukleidovské vzdálenosti:

```
qn = self.getNearestPoint(q, points)
```

Z těchto dvou bodů je vytvořena počáteční hrana:

```
e = Edge(q, qn)
es = Edge(qn, q)
```

Tyto hrany jsou vloženy do seznamu aktivních hran AEL. Algoritmus dále iteruje, dokud není AEL prázdný.

Pro každou hranu $e = (p_1, p_2)$ je hledán tzv. Delaunayovský bod p , který leží v levé polorovině vzhledem k hraně a maximalizuje úhel $\angle(p_1, p, p_2)$. Tento krok je implementován následovně:

```
if self.getPointLinePosition(p_i, p1, p2) == 1:
    phi = self.get2LinesAngle(p_i, p2, p_i, p1)
```

Po nalezení bodu jsou vytvořeny nové hrany:

```
e2 = Edge(e1s.getEnd(), p_dt)
e3 = Edge(p_dt, e1s.getStart())
```

Tyto hrany jsou přidány do triangulace a zároveň je aktualizován seznam AEL:

```
self.updateAEL(e2, AEL)
self.updateAEL(e3, AEL)
```


Funkce `updateAEL` zajišťuje odstranění hran s opačnou orientací:

```
if es in AEL:
    AEL.remove(es)
else:
    AEL.append(e)
```

4.2 Test polohy bodu vůči přímce

Základním geometrickým nástrojem algoritmu je test polohy bodu vůči orientované přímce, realizovaný pomocí vektorového součinu:

```
t = ux*vy - vx*uy
```

Kde $u = (b - a)$ je směrový vektor přímky, a $v = (p - a)$ je vektor od počátku přímky k testovanému bodu. Výsledek testu:

- $t > 0$ – bod leží vlevo,
- $t < 0$ – bod leží vpravo,
- $t = 0$ – bod leží na přímce.

4.3 Generování vrstevnic

Vrstevnice jsou konstruovány jako průnik trojúhelníkové sítě s horizontálními rovinami $z = h$

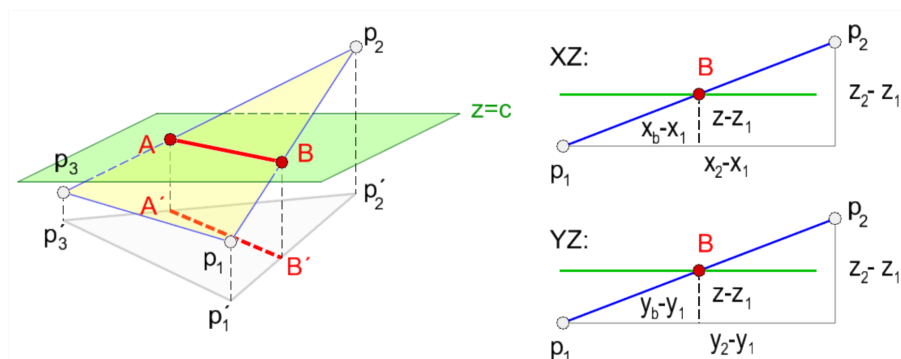
Algoritmus iteruje přes všechny výškové hladiny v intervalu $\langle z_{min}, z_{max} \rangle$ s krokem dz :

```
for z in range(z_min, z_max, dz):
```

Pro každý trojúhelník jsou testovány jeho hrany. Pokud rovina protíná hranu, je vypočten průsečík lineární interpolací:

```
xb = (p2.x() - p1.x())/(p2.z() - p1.z()) * (z - p1.z()) + p1.x()
yb = (p2.y() - p1.y())/(p2.z() - p1.z()) * (z - p1.z()) + p1.y()
```

Princip lineární interpolace použité při výpočtu průsečíku hrany s rovinou $z = h$ je znázorněn na obrázku:



Obrázek 4.1: Lineární interpolace

Výsledný bod je uložen jako:

```
QPoint3DF(xb, yb, z)
```

V případě, že jsou nalezeny dva průsečíky, je vytvořena úsečka vrstevnice:

```
e = Edge(a, b)
contour_lines.append(e)
```

4.4 Výpočet sklonu

Sklon trojúhelníku je definován jako úhel mezi jeho rovinou a horizontální rovinou.

Nejprve jsou definovány dva vektory a normálový vektor roviny. Výpočet v kódu je zkontruován následovně:

```
nx = uy*vz - uz*vy
ny = -(ux*vz - uz*vx)
nz = ux*vy - uy*vx
```

Sklon je následně určen jako:

```
return acos(nz/n)
```

kde n je velikost normálového vektoru.

4.5 Výpočet expozice

Expozice (orientace svahu) je určena projekcí normálového vektoru do roviny pomocí:

```
return atan2(ny, nx)
```

Tato metoda umožňuje správné určení směru v intervalu $\langle -\pi, \pi \rangle$.

4.6 Převod triangulace na trojúhelníky

Triangulace je reprezentována seznamem hran, přičemž každý trojúhelník je tvořen trojicí po sobě jdoucích hran.

Převod na objekty typu `Triangle` probíhá následovně:

```
for i in range(0, len(DT), 3):
    p1 = DT[i].getStart()
    p2 = DT[i+1].getStart()
    p3 = DT[i+2].getStart()
    triangle = Triangle(p1, p2, p3, 0, 0)
    triangles.append(triangle)
```

4.7 Analýza sklonu a expozice DMT

Analýza probíhá iterací přes všechny trojúhelníky, (analogicky je vypočtena i expozice).

```
for t in triangles:
    p1, p2, p3 = t.getVertices()
    slope = self.computeSlope(p1, p2, p3)
    t.setSlope(slope)
```

Kapitola 5

Problematické situace a ošetření těchto situací v kódu

5.1 Degenerace při Delaunay triangulaci

Jedním z hlavních problémů je degenerace vstupních dat, kdy body leží v nevhodné konfiguraci, například kolineární body, body ležící na jedné kružnici či velmi blízké body. V těchto případech může dojít k výběru nesprávného Delaunay bodu či numerickým chybám při výpočtu úhlů.

V implementaci je vliv numerických nepřesností při rozhodování o poloze bodu řešen zavedením tolerance při testu polohy bodu vůči přímce:

```
tolerance = 1.0e-6
if t > tolerance:
    return 1
if t < -tolerance:
    return 0
return -1
```

5.2 Neexistence Delaunay bodu

Při konstrukci triangulace může nastat situace, kdy pro danou hranu neexistuje žádný bod v levé polorovině. V takovém případě funkce vrací hodnotu `None`:

```
if p_dt == None:
    continue
```

Tato situace znamená, že hrana leží na konvexním obalu množiny bodů a nelze k ní vytvořit nový trojúhelník. Algoritmus proto pokračuje další hranou ze seznamu AEL.

5.3 Problémy s orientací hran

Správná orientace hran je klíčová, protože nesprávná orientace by vedla k chybnému určení levé poloroviny a nesprávnému výběru Delaunay bodu. V implementaci je orientace zajištěna metodou:

```
e1s = e1.switchOrientation()
```

Správná orientace je proto zajištěna vždy před voláním funkce `findDelaunayPoint`.

5.4 Duplicitní hrany v seznamu AEL

Při přidávání hran do seznamu aktivních hran může docházet k situaci, kdy se v seznamu nachází hrana i její opačně orientovaná varianta. To by vedlo k vytvoření duplicitních trojúhelníků. Tento problém je řešen funkcí `updateAEL`, která kontroluje existenci opačné hrany:

```
es = e.switchOrientation()
```

```
if es in AEL:
    AEL.remove(es)
else:
    AEL.append(e)
```

5.5 Dělení nulou při interpolaci

Při výpočtu průsečíku může dojít k dělení nulou, pokud $z_1 = z_2$. Tato situace je implicitně ošetřena tím, že se před interpolací kontrolují speciální případy (kolinearita).

5.6 Zjednodušení reprezentace triangulace

Triangulace je v implementaci reprezentována jako seznam hran, přičemž každý trojúhelník je tvořen trojicí po sobě jdoucích hran.

Převod je realizován následovně:

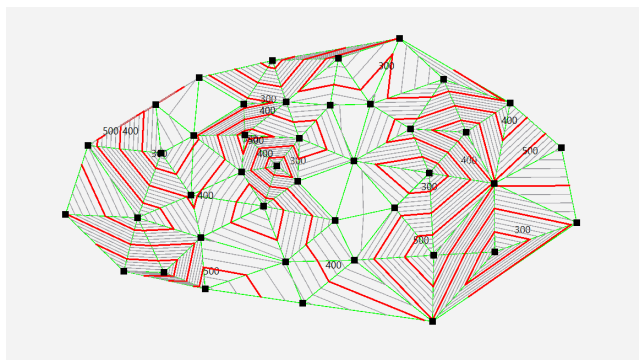
```
for i in range(0, len(DT), 3):
```

Tento přístup předpokládá, že každé tři po sobě jdoucí hrany tvoří jeden trojúhelník, což je nutné dodržet při konstrukci triangulace.

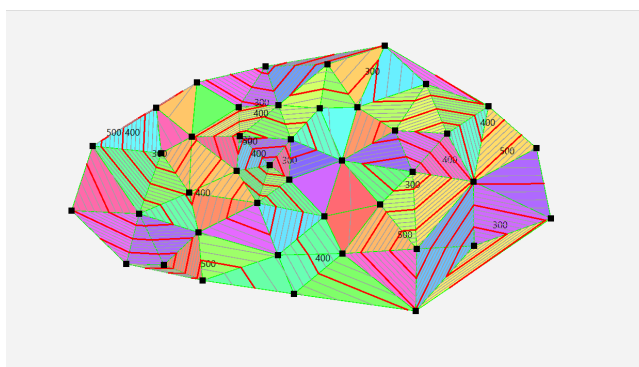
5.7 Problematické situace při generování syntetického terénu

Součástí aplikace je také možnost generování syntetických vstupních dat. Uživatel může body zadávat kliknutím do grafického okna, přičemž každému bodu je automaticky přiřazena výšková souřadnice z . V programu jsou implementovány tři režimy generování terénu: náhodný terén, hill a valley.

Výchozím režimem je náhodný terén:



Obrázek 5.1: Enter Caption



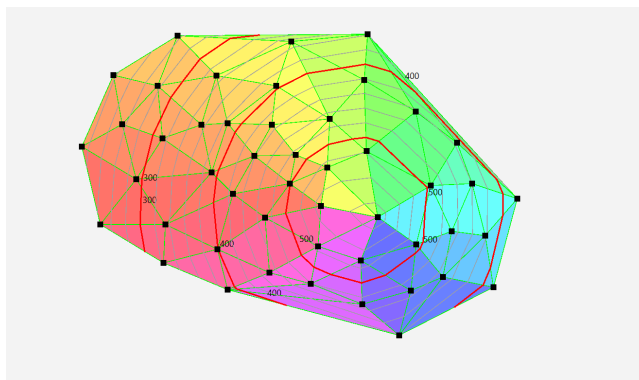
Obrázek 5.2: Enter Caption

```
z = random() * (z_max - z_min) + z_min
```

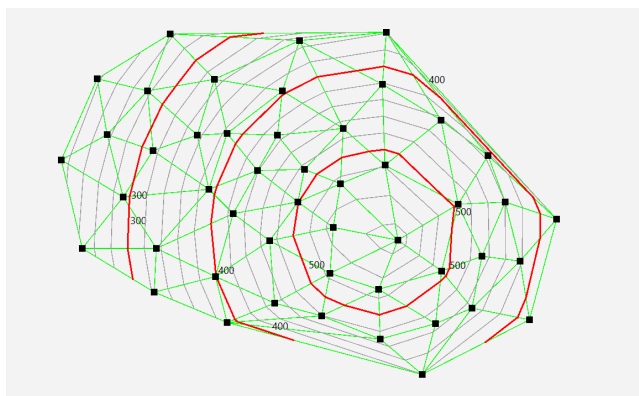
V tomto případě je výška bodu generována náhodně v intervalu od 200 do 600. Tento režim je vhodný pro základní testování algoritmů, avšak nemusí odpovídat reálnému reliéfu, protože sousední body mohou mít výrazně rozdílné výšky.

Druhým režimem je generování terénní kopce. Výška bodu je odvozena od vzdálenosti bodu od středu kreslicí plochy:

```
z = 600 - dist
```



Obrázek 5.3: Enter Caption



Obrázek 5.4: Enter Caption

Body blíže ke středu mají větší výšku, zatímco body vzdálenější od středu mají výšku nižší. Tím vzniká jednoduchý model vyvýšeniny.

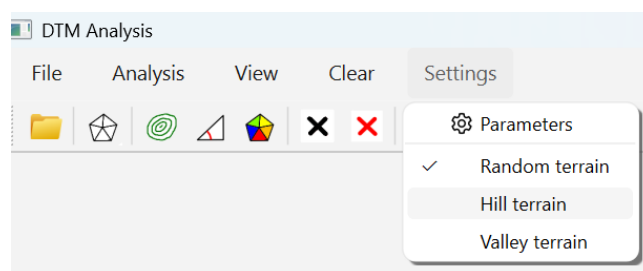
Třetím režimem je údolí:

$$z = 200 + \text{dist}$$

V tomto případě mají body ve středu nižší výšku a směrem od středu výška roste. Tím vzniká zjednodušený model údolní nebo snížené části terénu. Nevýhodou je, že výsledný tvar je silně idealizovaný a je závislý pouze na vzdálenosti od středu, nikoli na směru.

Volba režimu je nastavována pomocí metody:

```
def setTerrainMode(self, mode):
    self.__terrainMode = mode
```



Obrázek 5.5: Uživatelské rozhraní

Z hlediska problematických situací je důležité zmínit, že synteticky generovaný terén slouží především pro testování funkčnosti algoritmu. Nezohledňuje reálné geomorfologické procesy, povinné hrany, hřbetnice ani údolnice.

Kapitola 6

Vstupní data

Vstupní data pro tvorbu digitálního modelu terénu jsou reprezentována množinou prostorových bodů ve formátu:
 $P = \{(x_i, y_i, z_i)\}$

Aplikace umožňuje dva způsoby získání vstupních dat - interaktivní zadání bodů uživatelem a načtení bodů ze souboru.

6.1 Interaktivní zadávání bodů

Body mohou být zadávány přímo uživatelem kliknutím do grafického okna aplikace. Každé kliknutí myši vytvoří nový bod s automaticky přiřazenou výškou.

Získání souřadnic probíhá pomocí:

```
x = e.position().x()
y = e.position().y()
```

Výška bodu z je generována na základě zvoleného režimu terénu: random – náhodná výška, hill – model kopce, valley – model údolí.

Výsledný bod je vytvořen jako instance třídy:

```
p = QPoint3DF(x, y, z)
```

a následně uložen do seznamu vstupních bodů:

```
self.__points.append(p)
```

Tento způsob je vhodný především pro testování algoritmů a demonstraci chování modelu na syntetických datech.

6.2 Načítání dat ze souboru

Druhou možností je načtení dat ze souboru ve formátu XYZ, kde každý řádek obsahuje souřadnice jednoho bodu. Načtení souboru je realizováno pomocí dialogového okna:

```
res = QFileDialog.getOpenFileName(...)
```

Po načtení dat je provedena normalizace a přepoččet souřadnic do zobrazovacího prostoru aplikace. Nejprve jsou určeny minimální a maximální hodnoty:

```
min_x = min(p.x() for p in points)
max_x = max(p.x() for p in points)
```

Následně jsou body škálovány do rozsahu kreslicí plochy:

```
nx = 50 + (p.x() - min_x) / rx * W
ny = 50 + (max_y - p.y()) / ry * H
```

Tím je zajištěno zachování poměru stran, přizpůsobení dat velikosti okna a správná orientace osy y . Původní výškové hodnoty z zůstávají zachovány.

6.3 Správa vstupních dat

Načtená nebo vytvořená data jsou ukládána do seznamu bodů, který je přístupný metodou:

```
def getPoints(self):
    return self.__points
```

Při načtení nových dat dochází k vymazání původních bodů:

```
self.Canvas.getPoints().clear()
```

Tím je zajištěno, že se v aplikaci vždy pracuje pouze s jednou aktuální množinou vstupních dat.

Kapitola 7

Výstupní data

Výstupem aplikace nejsou strukturovaná data ukládaná do souboru, ale především grafická vizualizace digitálního modelu terénu (DMT) a z něj odvozených charakteristik. Aplikace slouží primárně jako nástroj pro interaktivní analýzu a interpretaci terénu.

Veškeré výsledky jsou zobrazovány přímo v grafickém okně aplikace. Výstup tedy představuje vizualizace Delaunay triangulace, vizualizace vrstevnic, vizualizace sklonu, vizualizace expozice.

7.1 Vizualizace Delaunay triangulace

Triangulace je zobrazena jako síť hran spojujících jednotlivé body. Hrany jsou vykreslovány pomocí:

```
qp.drawLine(edge.getStart(), edge.getEnd())
```

7.2 Vizualizace vrstevnic

Vrstevnice jsou zobrazeny jako spojitě úsečky reprezentující konstantní výšku. Pro lepší orientaci jsou rozlišeny hlavní vrstevnice (červené, silnější) a vedlejší vrstevnice (šedé, tenčí).

```
if int(z) % 100 == 0:
    pen_c = QPen(Qt.GlobalColor.red, 2)
else:
    pen_c = QPen(Qt.GlobalColor.gray, 1)
```

Pro zvýšení přehlednosti jsou hlavní vrstevnice doplněny textovými popisky nadmořské výšky. Popisky nejsou zobrazovány u každého segmentu, ale pouze u vybraných, aby nedocházelo k jejich překrývání.

Konkrétně je popisek vykreslen přibližně u každého sedmého segmentu vrstevnice:

```
if draw_label and i % 7 == 0:
    qp.drawText(int(xm + 3), int(ym - 3), label)
```

7.3 Vizualizace sklonu

Sklon terénu je vizualizován pomocí odstínů šedi, přičemž každému trojúhelníku je přiřazena barva podle jeho sklonu. Světlé odstíny odpovídají malému sklonu, tmavé odstíny reprezentují strmé svahy:

```
grey_val = int(255 - min(255, slope * 255 / (pi/2)))  
qp.setBrush(QColor(grey_val, grey_val, grey_val))
```

7.4 Vizualizace expozice

Expozice je zobrazena pomocí barevného modelu HSV, kde barva odpovídá směru orientace svahu. Různé barvy reprezentují různé azimuty, což umožňuje intuitivní interpretaci orientace svahů:

```
hue = int(((aspect + pi) / (2 * pi)) * 359)  
qp.setBrush(QColor.fromHsv(hue, 150, 255))
```

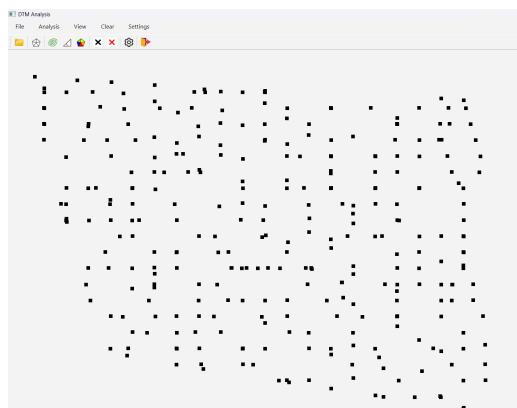
7.5 Shrnutí

Uživatel má možnost ovlivnit, které vrstvy budou zobrazeny. To je realizováno pomocí přepínačů. Jednotlivé vrstvy lze zapínat a vypínat nezávisle na sobě.

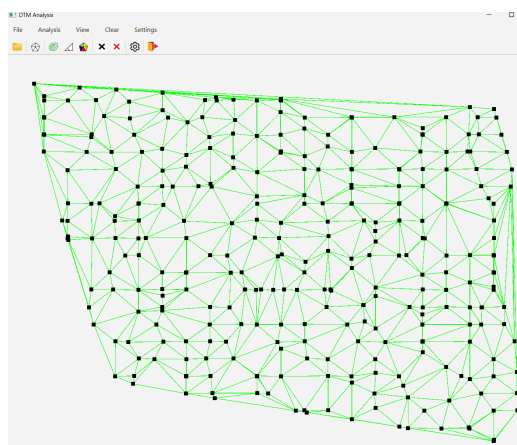
Výstup aplikace je čistě vizuální a slouží k interaktivní interpretaci digitálního modelu terénu. Kombinace triangulace, vrstevnic a morfometrických charakteristik poskytuje uživateli komplexní pohled na tvar a vlastnosti terénu, aniž by bylo nutné generovat externí datové soubory.

Kapitola 8

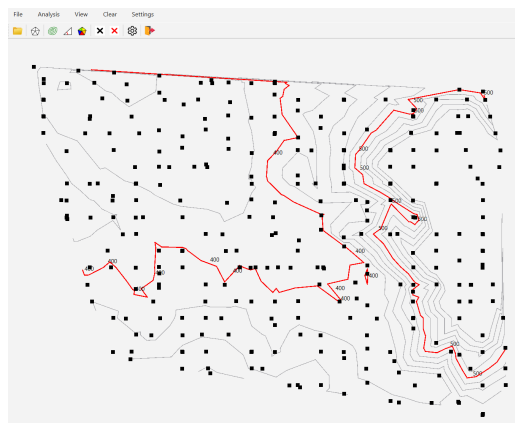
Printscreen vytvořené aplikace



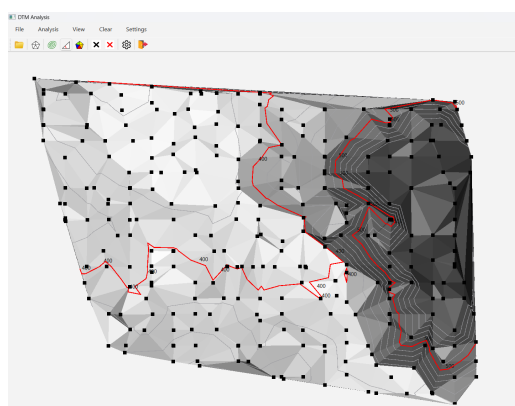
Obrázek 8.1: Testovací body



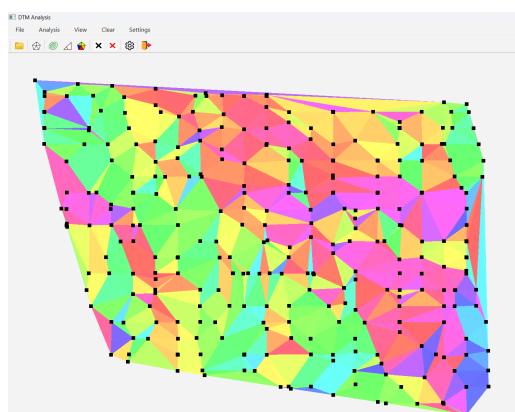
Obrázek 8.2: Vytvořená triangulace nad testovacími body



Obrázek 8.3: Vrstevnice nad testovacími body



Obrázek 8.4: Vizualizace sklonu nad testovacími body



Obrázek 8.5: Vizualizace expoze nad testovacími body

Kapitola 9

Dokumentace tříd, popis datových položek

9.1 Dokumentace třídy Algorithms.py

Tato třída implementuje klíčové geometrické algoritmy pro tvorbu digitálního modelu terénu (DTM), výpočet Delaunayho triangulace a následnou analýzu parametrů ploch (sklon, orientace).

Metody

- `getPointLinePosition(a, b, p)`
Určuje polohu bodu p vůči orientované úsečce ab pomocí testu poloroviny (vlevo, vpravo, na přímce).
- `getNearestPoint(p, points)`
Vyhledá v dané množině bodů ten, který se nachází v nejmenší euklidovské vzdálenosti od bodu p .
- `get2LinesAngle(p1, p2, p3, p4)`
Vypočítá úhel v radiánech mezi dvěma přímkami definovanými dvojicemi bodů s využitím skalárního součinu.
- `findDelaunayPoint(p1, p2, points)`
Hledá pro hranu $(p1, p2)$ optimální Delaunayho bod v levé polorovině, který maximalizuje úhel nad touto hranou.
- `createDT(points)`
Hlavní řídicí metoda, která generuje Delaunayho triangulaci metodou postupného připojování hran (Active Edge List).
- `updateAEL(e, AEL)`
Aktualizuje seznam aktivních hran - pokud hrana v seznamu existuje v opačné orientaci, odstraní ji, jinak ji přidá jako novou hranici.
- `getContourPoint(p1, p2, z)`
Provádí lineární interpolaci souřadnic x, y na hraně $(p1, p2)$ pro zadanou prahovou hodnotu z .
- `createContourLines(DT, z_min, z_max, dz)`
Vytváří soubor všech vrstevnic v zadaném rozsahu a kroku na základě prohledávání trojúhelníků sítě.

- `createContourLineSegment(p1, p2, p3, z, contour_lines)`
Vypočítá a přidá konkrétní úsečku vrstevnice procházející jedním trojúhelníkem.
- `computeSlope(p1, p2, p3)`
Vypočítá maximální sklon trojúhelníkové plochy vůči horizontální rovině.
- `computeAspect(p1, p2, p3)`
Určuje expozici (orientaci) svahu, tedy směr, kam je plocha trojúhelníka natočena vzhledem ke světovým stranám.
- `convertDTToTriangles(DT, triangles)`
Konvertuje topologická data (seznam hran) na objekty typu `Triangle` pro snazší analýzu.
- `analyzeDTMSlope(DT, triangles)`
Iteruje přes všechny trojúhelníky modelu a přiřazuje jim vypočtenou hodnotu sklonu.
- `analyzeDTMAsspect(DT, triangles)`
Iteruje přes všechny trojúhelníky modelu a přiřazuje jim vypočtenou hodnotu orientace (aspektu).

9.2 Dokumentace třídy `Ui_MainWindow`

Tato třída definuje hlavní uživatelské rozhraní aplikace vytvořené v knihovně PyQt6. Zajišťuje vykreslování grafických prvků, ovládání menu a propojení uživatelských akcí s výpočetními algoritmy.

Datové položky

Obsahuje prvky uživatelského rozhraní (menu, toolbar, canvas).

Metody

- `setupUi(MainWindow)`
Provádí inicializaci grafických komponent (plátno `Canvas`, menu, nástrojová lišta) a propojuje události tlačítek (triggered) s příslušnými metodami.
- `openClick()`
Otevírá dialog pro výběr souboru `.xyz`, načítá body a provádí jejich normalizaci a transformaci souřadnic, aby se korektně vykreslily na plátno aplikace.
- `createDTClick()`
Spouští výpočet Delaunayho triangulace nad načtenými body a předává výsledek plátnu k vykreslení.
- `createContourLinesClick()`
Generuje vrstevnice pro zadaný výškový rozsah a krok. Pokud triangulace neexistuje, automaticky ji před výpočtem vytvoří.
- `analyzeSlopeClick()`
Spouští analýzu sklonu svahů pro jednotlivé trojúhelníky a aktivuje jejich barevné zobrazení na plátně.
- `analyzeExpositionClick()`
Provádí analýzu orientace (expozice) ploch vůči světovým stranám a aktualizuje zobrazení v aplikaci.

- `retranslateUi(MainWindow)`
Zajišťuje textové popisky ovládacích prvků, zkratky a nástrojové nápovědy v uživatelském prostředí.

9.3 Dokumentace třídy Draw

Tato třída slouží pro vykreslování všech grafických výstupů aplikace a zároveň pro interaktivní zadávání vstupních bodů uživatelem.

Datové položky

- `__points` – seznam vstupních bodů
- `__DT` – seznam hran Delaunay triangulace
- `__contours` – seznam vrstevnic
- `__triangles` – seznam trojúhelníků pro analýzu
- `__viewDT` – přepínač zobrazení triangulace
- `__viewContours` – přepínač zobrazení vrstevnic
- `__viewSlope` – přepínač zobrazení sklonu
- `__viewAspect` – přepínač zobrazení expozice
- `__terrainMode` – režim generování terénu (random, hill, valley)

Metody

- `mousePressEvent(e)`
Zachycuje kliknutí myši, kterým uživatel přidává nový bod do scény. Každému bodu je automaticky přiřazena náhodná nadmořská výška z pro účely testování.
- `paintEvent(e)`
Centrální vykreslovací metoda. Postupně vykresluje:
 - Barevné plochy trojúhelníků (stínovaný sklon v šedi nebo orientace v HSV barvách).
 - Síť Delaunayho triangulace (zeleně).
 - Vrstevnice (šedě, zvýrazněné hlavní vrstevnice červeně).
 - Vstupní body (černé body).
- `setViewSlope / setViewExposition(state)`
Přepínají mezi režimy zobrazení analýz. Tyto režimy jsou vzájemně výlučné (zapnutí jednoho vypne druhý).
- `setDT / setContours / setTriangles(data)`
Setter metody, které slouží k předání vypočtených geometrických struktur z třídy `Algorithms` na plátno.
- `clearResult() / clearAll()`
Metody pro promazání buď pouze vypočtených analýz (triangulace, vrstevnice), nebo úplné vyčištění plátna včetně vstupních bodů.

9.4 Dokumentace třídy Triangle

Tato třída reprezentuje jednotlivé trojúhelníky v síti DTM a uchovává jejich geometrické i morfometrické vlastnosti.

Datové položky

- `__p1`, `__p2`, `__p3` – vrcholy trojúhelníku
- `__slope` – hodnota sklonu
- `__aspect` – hodnota expozice

Metody

- `__init__(p1, p2, p3, slope, aspect)`
Konstruktor třídy, který inicializuje vrcholy trojúhelníka (objekty `QPoint3DF`) a volitelně nastavuje jeho sklon a orientaci.
- `getVertices()`
Vrací trojici vrcholů `p1, p2, p3`, což je klíčové pro vykreslování polygonů a výpočty normálových vektorů.
- `getSlope()` / `setSlope(slope)`
Getter a setter pro hodnotu sklonu plochy trojúhelníka (v radiánech).
- `getAspect()` / `setAspect(aspect)`
Getter a setter pro hodnotu orientace (aspektu) plochy vůči světovým stranám.

9.5 Dokumentace třídy Edge

Tato třída definuje orientovanou hranu v grafické scéně, která propojuje dva body. Je základním stavebním prvkem pro triangulační síť i vrstevnice.

Datové položky

- `__start` – počáteční bod
- `__end` – koncový bod

Metody

- `__init__(p1, p2)`
Konstruktor inicializující počáteční (`start`) a koncový (`end`) bod hrany.
- `getStart()` / `getEnd()`
Metody pro přístup k hraničním bodům úsečky.

- `switchOrientation()`
Vytvoří a vrátí novou instanci hrany se zaměněným pořadím bodů. Tato operace je klíčová pro algoritmus Delaunayho triangulace při práci se seznamem aktivních hran (AEL).
- `__eq__(other)`
Přeformátovaný operátor rovnosti, který umožňuje porovnávat dvě hrany. Dvě hrany jsou považovány za shodné, pokud mají identické počáteční i koncové body ve stejném pořadí.

9.6 Dokumentace třídy `QPoint3DF`

Datové položky

- `__z` – výška bodu

Metody

Tato třída je potomkem standardní třídy `QPointF`. Rozšiřuje její funkcionalitu o třetí rozměr (souřadnici z), což umožňuje reprezentaci bodů v trojrozměrném prostoru (3D).

- `__init__(x, y, z)`
Konstruktor třídy, který přebírá souřadnice x a y pro základní třídu `QPointF` a nově definuje privátní atribut pro výšku z .
- `z()`
Getter metoda, která vrátí výškovou souřadnici bodu. Tato metoda je klíčová pro výpočty sklonu, orientace a pro generování vrstevnic.

Kapitola 10

Závěr

Cílem práce bylo vytvořit aplikaci pro konstrukci digitálního modelu terénu na základě nepravidelně rozmístěných bodů a následně provést základní analýzy terénu. Model terénu byl reprezentován pomocí triangulované nepravidelné sítě (TIN), vytvořené na základě Delaunay triangulace.

V rámci práce byly implementovány algoritmy pro konstrukci triangulace, generování vrstevnic a výpočet sklonu a expozice. Výsledky jsou prezentovány prostřednictvím interaktivní grafické aplikace, která umožňuje vizuální analýzu terénu.

Navržené řešení umožňuje jak načítání reálných dat ze souboru, tak generování syntetického terénu. To usnadňuje testování algoritmů a demonstraci jejich vlastností. Vizualizace jednotlivých výstupů, jako jsou vrstevnice, sklon nebo expozice, poskytuje uživateli komplexní pohled na strukturu terénu.

Neřešené problémy

V rámci implementace zůstalo několik problematických oblastí, které nebyly dále řešeny a mohou ovlivnit kvalitu výsledného digitálního modelu terénu. Jedním z hlavních omezení je skutečnost, že model nezohledňuje přirozené geomorfologické struktury terénu, jako jsou hřbetnice a údolnice. Tyto linie hrají důležitou roli při interpretaci reliéfu, avšak v rámci použité metody nejsou explicitně identifikovány ani zachovány. Výsledný model tak představuje spíše geometrickou aproximaci povrchu než jeho plnohodnotnou geomorfologickou reprezentaci.

Určité zjednodušení představuje také způsob reprezentace triangulace, která je ukládána pouze jako seznam hran bez explicitního popisu topologických vazeb mezi trojúhelníky. Tento přístup je sice implementačně jednoduchý, avšak omezuje možnosti dalšího zpracování a analýzy dat.

V neposlední řadě aplikace neumožňuje export výsledků do externího formátu, což omezuje její využitelnost v širším kontextu geografických informačních systémů. Výstupy jsou dostupné pouze ve formě vizualizace v rámci aplikace. Dalším omezením je práce pouze ve 2D prostředí, bez prostorového zobrazení modelu terénu, což může ztěžovat interpretaci složitějších tvarů reliéfu.

Možnosti vylepšení

Navržené řešení nabízí řadu možností dalšího rozšíření a vylepšení. Jedním z hlavních směrů je implementace constrained Delaunay triangulace, která by umožnila zachování významných linií terénu, jako jsou

hřbetnice nebo údolnice, a tím zvýšila kvalitu výsledného modelu.

Dalším logickým krokem je rozšíření aplikace o 3D vizualizaci, která by umožnila realističtější zobrazení terénu a lepší interpretaci jeho prostorových vlastností. Vylepšení by si zasloužila také samotná vizualizace, například zavedením plynulého stínování nebo pokročilejších barevných škál pro znázornění sklonu a expozice.

Z hlediska uživatelského komfortu by bylo vhodné automatizovat některé kroky analýzy, například spouštění výpočtů bez nutnosti manuálního zásahu uživatele. Dále by bylo přínosné rozšířit podporu vstupních dat o další formáty, například GeoJSON, a umožnit práci s více datovými vrstvami současně.

V oblasti analýzy terénu by bylo možné doplnit další charakteristiky, jako je zakřivení povrchu, hydrologické analýzy nebo identifikace významných liniových struktur. Tím by se aplikace posunula od demonstračního nástroje k plnohodnotnějšímu analytickému prostředí.

Kapitola 11

Použitá literatura

BAYER, T. (2026): Algoritmy počítačové kartografie, Rovinné triangulace a jejich využití . Přírodovědecká fakulta, Karlova Univerzita.

PyQt6 Documentation (2024).

Dostupné z: <https://www.riverbankcomputing.com/static/Docs/PyQt6/>

BOURKE, P. (1997): Interpolation on a Triangle.

Dostupné z: <http://paulbourke.net/miscellaneous/interpolation/>